Laboratorul 9. Colecții: liste, tupluri, mulțimi, dicționare

Privesc pe Linux: enumerare spre root

fișă de laborator · 2 × 90 min (cu pauză la mijloc) · două lucrări · se lucrează individual, la terminal

> **CE FACEȚI ASTĂZI.** Construiți o unealtă de enumerare spre root: enum.py citește ce a adunat un atacator pe o mașină Linux (fișiere SUID, reguli sudo, sarcini cron), așază totul în colecțiile potrivite și scoate un raport JSON (Lucrarea A). Apoi comparați sistemul cu o imagine curată și scoateți, cu mulțimi și comprehensions, exact ce e nou și periculos (Lucrarea B). Nu aveți nevoie de o mașină reală: capturile vă sunt date.

> **CUM LUCRAȚI (deep-end).** Fișa vă dă obiectivul, scheletul cu # TODO și comanda de rulare, nu rândul gata scris. Scrieți voi codul. După fiecare pas rulați și comparați cu caseta verde „Pe ecran trebuie să vezi". Explicația de programare (liste, tupluri, mulțimi, dicționare, comprehensions) e în suportul de curs; aici o puneți în practică.

Fișierele de start

Din secțiunea sesiunii de pe ELSE descărcați folderul start/ și puneți-l în depozitul vostru de curs, tehnici-prog-<nume>, sub arsenal/privesc/. Conține capturile de pe o mașină compromisă:

Fișier	Ce conține (cum s-ar obține pe o mașină reală)
`suid.txt`	fișiere cu bit SUID: find / -perm -4000 -type f 2>/dev/null
`sudo.txt`	ce poate rula contul fără/cu parolă: sudo -l
`crontab.txt`	sarcini programate de sistem: cat /etc/crontab
`baza_suid.txt`	aceleași SUID pe o instalare curată, de referință (pentru Lucrarea B)

> **DE CE DIN FIȘIERE.** Pe mașina reală, comenzile de mai sus scot sute de rânduri. Le-am capturat o dată în fișiere ca lucrarea să fie aceeași pentru toți și să ruleze pe orice calculator. Unealta voastră citește fișierele; pe o țintă reală, ați înlocui citirea din fișier cu rularea comenzii. Datele sunt de la o mașină de laborator, izolată; nu rulați enumerare pe sisteme care nu vă aparțin.

LUCRAREA A (nucleu). enum.py: adună și grupează

Obiectiv: arsenal/privesc/enum.py citește cele trei capturi, așază constatările în colecții și scrie un raport JSON plus un rezumat pe categorii. Fiecare pas folosește o altă colecție.

A1. Scheletul și prima listă (15 min)

Creează arsenal/privesc/enum.py. Întâi citești fișierele SUID într-o listă și o numeri. Un fișier de citire comun îți scoate liniile curate (fără goluri, fără comentarii):

```
import argparse, json, os

def citeste_linii(cale):
    with open(cale, encoding="utf-8") as f:
        # TODO: intoarce lista liniilor .strip(), fara cele goale si fara cele cu #
        ...

cai = citeste_linii("start/suid.txt")
print("linii SUID:", len(cai))
```

Pe ecran trebuie să vezi:

```
linii SUID: 14
```

De observat. Sunt 14 rânduri, dar unele se repetă. Lista păstrează tot, în ordine, inclusiv dublurile. Le strângem la pasul următor.

A2. Nume unice: de la listă la mulțime (12 min)

Numele fișierului e ce decide dacă e periculos (find, vim, nmap...). Scoate numele din fiecare cale și ține-le o singură dată.

```
def nume(cale):
    # TODO: intoarce ce e dupa ultimul "/"  (vezi rsplit)
    ...

nume_unice = {nume(c) for c in cai}    # set comprehension
print("SUID distincte:", len(nume_unice))
```

Pe ecran trebuie să vezi:

```
SUID distincte: 13
```

De observat. 14 rânduri, 13 nume distincte: mulțimea a aruncat dublura. Dacă îți dă tot 14, verifică funcția nume: probabil întoarce toată calea, nu doar numele.

A3. Constatările SUID: listă de dicționare (18 min)

Fiecare SUID devine o fișă: calea, numele, riscul. Periculos = numele e într-o mulțime de binare cunoscute ca scăpări (stil GTFOBins). Întorci o listă de dicționare, sortată, fără dubluri.

```
GTFO = {"find", "vim.basic", "vim", "nmap", "cp", "bash",
        "less", "awk", "python3", "perl"}

def suid_constatari(cale):
    cai = citeste_linii(cale)
    # TODO: pentru fiecare cale UNICA, in ordine sortata, un dict:
    #   {"cale": c, "nume": nume(c),
    #    "risc": "mare" daca nume(c) in GTFO, altfel "normal"}
    ...

suid = suid_constatari("start/suid.txt")
mari = [c for c in suid if c["risc"] == "mare"]
print(len(suid), "SUID,", len(mari), "cu risc mare")
print(sorted(c["nume"] for c in mari))
```

Pe ecran trebuie să vezi:

```
13 SUID, 4 cu risc mare
['cp', 'find', 'nmap', 'vim.basic']
```

A4. sudo și cron: tuplul și gruparea (20 min)

O linie din sudo.txt are forma „user : drept". O despachetezi într-un tuplu. O linie din crontab.txt are șase câmpuri de program, apoi userul și comanda. Fiecare sursă dă o listă de dicționare.

```
def sudo_constatari(cale):
    out = []
    for ln in citeste_linii(cale):
        user, drept = ln.split(" : ", 1)   # despachetare in tuplu
        # TODO: adauga {"user": user.strip(), "drept": drept.strip(),
        #   "risc": "mare" daca "NOPASSWD" in drept, altfel "normal"}
        ...
    return out

def cron_constatari(cale):
```

```
    # TODO: campuri = ln.split(); user = campuri[5];
    #   comanda = " ".join(campuri[6:]); dict cu user si comanda
    ...
```

Acum le grupezi pe toate într-un singur raport: un dicționar cu o cheie pe categorie, valoarea = lista ei.

```
def enumereaza(d):
    return {
        "suid": suid_constatari(os.path.join(d, "suid.txt")),
        "sudo": sudo_constatari(os.path.join(d, "sudo.txt")),
        "cron": cron_constatari(os.path.join(d, "crontab.txt")),
    }
```

A5. Raportul: JSON și rezumat pe categorii (15 min)

main() cu argparse: primește folderul capturilor și un comutator --json. Cu --json scoate JSON curat; fără el, un rezumat pe categorii (cu .items()) și salvează raportul pe disc.

```
def main():
    p = argparse.ArgumentParser()
    p.add_argument("capturi")
    p.add_argument("--json", action="store_true")
    a = p.parse_args()
    raport = enumereaza(a.capturi)
    if a.json:
        print(json.dumps(raport, indent=2, ensure_ascii=False)); return
    for categorie, lista in raport.items():
        mari = [c for c in lista if c.get("risc") == "mare"]
        print(f"{categorie}: {len(lista)} gasite, {len(mari)} cu risc mare")
    # TODO: utilizatorii implicati = multimea userilor din sudo REUNITA cu cei din cron
    with open("enum.json", "w", encoding="utf-8") as f:
        json.dump(raport, f, indent=2, ensure_ascii=False)
    print("Raport salvat in enum.json")

if __name__ == "__main__":
    main()
```

Rulezi din rădăcina depozitului, cu -m, dând calea capturilor:

```
python -m arsenal.privesc.enum arsenal/privesc/start
```

Pe ecran trebuie să vezi:

```
suid: 13 gasite, 4 cu risc mare
sudo: 3 gasite, 2 cu risc mare
cron: 3 gasite, 0 cu risc mare
utilizatori implicati: ana, root, www-data
Raport salvat in enum.json
```

> **DE REȚINUT.** Patru colecții, fiecare la locul ei: lista ține fișierele în ordine, tuplul despachetează o linie, mulțimea scoate userii unici, dicționarul grupează raportul. Codul a ieșit scurt pentru că structura a fost aleasă bine.

LUCRAREA B (extindere). Ce e nou față de un sistem curat

Obiectiv: un apărător compară mașina de azi cu o imagine curată. Tu faci la fel, cu diferența de mulțimi: scoți exact fișierele SUID apărute și, dintre ele, pe cele periculoase. Lucrezi în același folder, într-un al doilea fișier, arsenal/privesc/diferenta.py.

B1. Două mulțimi: curat și acum (15 min)

```
def cai(fisier):
    # TODO: intoarce MULTIMEA cailor din fisier (foloseste citeste_linii din enum)
    ...
```

```
baza = cai("start/baza_suid.txt")
acum = cai("start/suid.txt")
print("baza:", len(baza), " acum:", len(acum))
```

Pe ecran trebuie să vezi:

```
baza: 9  acum: 13
```

B2. Diferența: ce a apărut (15 min)

Ce e în „acum" dar nu în „bază" sunt fișierele SUID noi. Un singur operator, nu o buclă dublă.

```
noi = acum - baza
print(len(noi), "SUID noi fata de sistemul curat")
for c in sorted(noi):
    print(" ", c)
```

Pe ecran trebuie să vezi:

```
4 SUID noi fata de sistemul curat
  /bin/cp
  /usr/bin/find
  /usr/bin/nmap
  /usr/bin/vim.basic
```

B3. Dicționar de risc și filtrul final (20 min)

Faci o fișă nume -> risc pentru cele noi (dict comprehension), apoi filtrezi doar periculoasele (list comprehension cu if). Refolosești nume() și GTFO din enum printr-un import.

```
from arsenal.privesc.enum import nume, GTFO

risc = {nume(c): ("mare" if nume(c) in GTFO else "normal") for c in noi}
periculoase = [c for c in sorted(noi) if nume(c) in GTFO]
print("periculoase NOI:", periculoase)
```

Pe ecran trebuie să vezi:

```
periculoase NOI: ['/bin/cp', '/usr/bin/find', '/usr/bin/nmap', '/usr/bin/vim.basic']
```

De observat. Toate cele patru apărute sunt periculoase: cineva a pus bit SUID pe find, nmap, cp și vim. Oricare dă root. Asta e exact genul de constatare pe care o scoți în față, nu o îngropi în raport.

B4. Un test care prinde regresia (15 min)

Scrie arsenal/privesc/test_diferenta.py cu un test mic, cu date scrise de mână, care verifică faptul cheie: ce e în acum și nu în bază iese corect. Rulează cu pytest și treacă fără rețea.

```
def test_doar_ce_e_nou():
    acum = {"a", "b", "c"}
    baza = {"a", "b"}
    assert acum - baza == {"c"}
```

```
pytest -q arsenal/privesc/test_diferenta.py
```

Pe ecran trebuie să vezi:

```
1 passed
```

OPȚIONAL (pentru vârful sălii sau acasă)

1. --risc mare: un comutator care scoate din raport doar constatările cu risc mare, din toate categoriile, printr-un comprehension.

2. enum.py pe o țintă reală: înlocuiește citirea din fișier cu subprocess.run(["find", "/", "-perm", "-4000"], ...) pe mașina Kali/Metasploitable din laborator, izolată. Compari cu capturile date.

3. Grupare după risc: un dicționar risc -> listă de nume, construit cu .setdefault, din toate categoriile deodată.

Ce predați și cum se notează

Predarea e prin git, în depozitul vostru tehnici-prog-<nume>. Nimic pe e-mail, nicio captură de ecran.

```
git add arsenal/privesc
git commit -m "privesc: enum SUID/sudo/cron -> JSON, diferenta fata de baza"
git push
```

Livrabil	Ce conține
arsenal/privesc/enum.py	citește capturile, grupează pe categorii, scoate rezumat și JSON (Lab A).
arsenal/privesc/enum.json	raportul generat: dicționar de liste de dicționare.
arsenal/privesc/diferenta.py	diferența de mulțimi față de sistemul curat, filtrul periculoaselor (Lab B).
arsenal/privesc/test_diferenta.py	un test care trece cu pytest (Lab B).

Se punctează: alegerea colecției potrivite (listă/tuplu/mulțime/dicționar, nu variabile separate), .get folosit unde o cheie poate lipsi, comprehensions corecte și un JSON valid. Asistentul vă poate cere să explicați de ce ați pus o anume valoare într-o listă și nu într-o mulțime.